



Machine Learning Operations

Final Team Project

End-to-End MLOps Pipeline

From Raw Data to a Monitored Production System

Module: DDSC611 – Machine Learning Engineering Practices

Instructor: Mohamed Tharwat, PhD, SM IEEE

Team Size: 3 Students

Submission Deadline: Friday, 8 May 2026 – 11:59 PM

Final Discussion: Sunday, 10 May 2026

Project Weight: 40% of Final Course Grade

Academic Year: 2025–2026 (Spring)

Important Notice

This is a **team-based project**. All three members are expected to contribute meaningfully and to understand the **entire pipeline end-to-end**. The final discussion on May 13 will include **individual questions** directed at each team member. Inability to explain code written by a teammate will affect your individual score.

Contents

1	Project Overview	2
1.1	Purpose	2
1.2	Learning Objectives	2
1.3	Pipeline Architecture	2
2	Dataset Selection	3
2.1	Requirements	3
2.2	Recommended Datasets	3
3	Mandatory Pipeline Components	5
3.1	Component 1: Data Versioning with DVC	5
3.2	Component 2: Preprocessing Pipeline	5
3.3	Component 3: Experiment Tracking & Model Registry	6
3.4	Component 4: Model Serving Application	7
3.5	Component 5: CI/CD Pipeline with GitHub Actions	7
3.6	Component 6: Monitoring & Drift Detection	8
3.7	Component 7: Documentation	8
3.8	Component 8: Project Setup & Reproducibility	9
4	Bonus Components	10
5	Deliverables & Submission	11
5.1	What to Submit	11
5.2	Required Repository Structure	11
6	Grading Rubric	13
6.1	Component Score Breakdown	13
6.2	Quality Scale	15
6.3	Individual vs. Team Score	15
7	Team Collaboration & Academic Integrity	16
7.1	Team Formation & Registration	16
7.2	Collaboration Guidelines	16
7.3	Academic Integrity	16
8	Technical Guidance	17
8.1	Getting-Started Checklist	17
8.2	Common Pitfalls	17
8.3	Quick Reference: Key Commands	18

1 Project Overview

1.1 Purpose

This final project is the capstone of DDSC611. It challenges teams to build a **production-grade MLOps pipeline** from scratch, applying every major concept covered across the ten lectures. The objective is not to achieve the highest possible model accuracy, but to build a system that is **reproducible, testable, deployable, and observable** — the hallmarks of mature MLOps practice.

“Deploying a model is not the end of the work. It is the beginning.”

Your pipeline must treat data, code, models, and infrastructure as **first-class versioned artifacts**. A model that cannot be reproduced, monitored, and updated on new data is not production-ready, regardless of its offline accuracy.

1.2 Learning Objectives

Upon successful completion, each team member will be able to demonstrate:

1. Design and implement a complete end-to-end MLOps pipeline
2. Apply version control to code (Git), data (DVC), and models (MLflow Registry)
3. Build reproducible, containerised environments using Docker and Docker Compose
4. Automate testing and validation with a GitHub Actions CI/CD pipeline
5. Serve a production model through a validated REST API (FastAPI)
6. Monitor a live system for data drift, model degradation, and infrastructure health
7. Produce professional documentation including a Model Card and a Data Card

1.3 Pipeline Architecture

The four stages below map directly to the mandatory components in Section 3. The drift detection stage feeds back into the data stage when distribution shift is detected, triggering a retraining cycle.

Stage	Name	Key Steps
1	Data	Raw data → DVC versioning → Preprocessing pipeline → Feature engineering
2	Training	Hyperparameter tuning → MLflow tracking → Model Registry → Validation
3	Deployment	Docker containerisation → CI/CD (GitHub Actions) → FastAPI serving
4	Monitoring	Evidently drift reports → Prometheus metrics → Alert & retrain loop

2 Dataset Selection

2.1 Requirements

Your team must select **one dataset** for the entire project satisfying all of the following criteria:

- **Tabular or structured** — time-series data is acceptable and encouraged
- **Publicly available** with a clear source URL and an open license
- **Sufficient size** — at least 10,000 rows to support meaningful preprocessing, hyperparameter tuning, and drift simulation
- **Splittable into reference and production sets** — either by time, by cohort, or by synthetic perturbation

Drift Simulation Requirement

You **must** demonstrate data drift for the monitoring component. Acceptable approaches:

- Split by time period (first 70% = reference, remainder = production)
- Apply distribution shifts to a held-out set (scale perturbation, subgroup filtering)
- Introduce synthetic noise to selected features to simulate sensor drift

The simulation must be **documented** and its effect clearly visible in the Evidently report.

2.2 Recommended Datasets

Dataset	Task	Key Challenge	Why Recommended
NYC Yellow Taxi (TLC Open Data)	Regression	Temporal drift, geo-features	Natural train/production split by month; seasonal drift
IEEE-CIS Fraud (Kaggle)	Binary Classification	Class imbalance ($\approx 3.5\%$ fraud)	590K rows, 400+ features; concept drift inherent
Air Quality UCI (UCI ML Repo)	Regression	Missing values, sensor drift	Realistic IoT sensor data; progressive degradation
Telco Churn (IBM / Kaggle)	Binary Classification	Mixed types, concept drift	Business-level drift simulation; fairness testing
Bike Sharing (UCI / Kaggle)	Regression	Seasonality, feature interactions	Train on one season, monitor on another
Adult Income (UCI ML Repo)	Binary Classification	Bias, fairness, sensitive attributes	Classic fairness dataset; drift by subgroup filtering
Home Credit Default (Kaggle)	Binary Classification	High dimensionality, heavy missing data	Most complex option; 300K rows, 120+ features

If your team wishes to use a dataset **not on the list above**, submit a **one-paragraph written justification** to me by **24 April 2026**. The justification must explain how the dataset meets the drift simulation requirement and covers sufficient preprocessing and modelling complexity.

3 Mandatory Pipeline Components

Your project must implement **all eight components** below. Failure to implement a component results in zero points for that component.

3.1 Component 1: Data Versioning with DVC

- Initialise a DVC repository linked to Git (`dvc init`)
- Track **at least three artifacts**: raw data, cleaned data, and train/test split
- Configure a **DVC remote** (a local directory path is acceptable)
- Implement a **DVC pipeline** (`dvc.yaml`) with at minimum three stages: `prepare`, `preprocess`, and `featurize`
- `dvc repro` on a clean checkout must regenerate all artifacts **deterministically** with the same `dvc.lock` hash values

Evidence: `dvc.yaml`, `dvc.lock` committed to Git, screenshot of `dvc dag` in the report.

3.2 Component 2: Preprocessing Pipeline

- A **scikit-learn Pipeline** encapsulating all preprocessing steps as a single serialisable object
- Must include: missing value imputation, feature scaling, and categorical encoding
- Must include **at least one** of: SMOTE, feature selection, or polynomial terms
- Fitted pipeline **saved as a DVC-tracked artifact** (eliminates train/serve skew)
- All parameters defined in `configs/params.yaml` — no hardcoded values

Evidence: Preprocessing code, saved artifact, `params.yaml`, and **unit tests for at least three transformation functions**.

3.3 Component 3: Experiment Tracking & Model Registry

- **MLflow Tracking Server** running as a Docker service (UI at `http://localhost:5000`)
- Log for **every run**: all hyperparameters, loss curves, test metrics, and the serialised model artifact
- Conduct **at least three experiments** varying algorithm, hyperparameters, or feature set
- Apply **automated HPO**: Grid Search, Randomised Search, or Optuna
- Register the best model and transition through **None** → **Staging** → **Production** via the MLflow API

Evidence: MLflow UI screenshot (runs comparison view), model registry screenshot showing Production model, and the Python stage-transition code.

3.4 Component 4: Model Serving Application

Teams may use **Flask** (simpler) or **FastAPI** (recommended) to serve their trained model.

- A web application that loads the trained model at startup (from MLflow Registry or from a saved artifact file)
- Two required endpoints:
 - GET `/health` — returns model name/version and status
 - POST `/predict` — accepts a JSON record and returns a prediction
- **Optional for full marks:** a POST `/predict/batch` endpoint accepting a list of records and returning predictions with confidence scores
- Basic input validation (type checking and required fields); Pydantic schemas are encouraged for FastAPI but not mandatory for Flask
- A short **test script** (`tests/test_api.py`) sending sample requests and asserting correct responses

Evidence: Serving application source code, screenshot of a working `/predict` call, and the test script with passing results.

3.5 Component 5: CI/CD Pipeline with GitHub Actions

A `.github/workflows/ci.yml` that triggers on every push to `main` and on every pull request, executing these stages **in order**:

1. **Lint** — code style check with `flake8` or `ruff`
2. **Unit Tests** — `pytest` with coverage report; pipeline fails if coverage drops below **70%**
3. **Data Validation** — schema and quality checks (Great Expectations or Pandera)
4. **Model Validation** — load the registered model; assert it meets a defined minimum performance threshold on the test set

At least **one branch protection rule** on `main` requiring all CI checks to pass before a PR can be merged.

Evidence: `.github/workflows/ci.yml`, screenshot of a **green** CI run, and the `pytest` coverage report.

3.6 Component 6: Monitoring & Drift Detection

- A **monitoring script** (`monitoring/run_monitoring.py`) comparing a reference dataset against a simulated production set with injected drift
- **Two Evidently HTML reports:**
 1. **Baseline report** — reference vs. clean held-out set (minimal drift)
 2. **Drift report** — reference vs. perturbed set (drift on ≥ 3 features)
- Both reports must include: Data Drift Report, Data Quality Report, and (if ground truth available) a Model Performance Report
- **Threshold logic:** if drift detected on $>20\%$ of features, print a structured warning and write a log entry listing drifted features and scores
- **Five custom Prometheus metrics:** confidence histogram, feature histograms for two key inputs, current model version (gauge), and inference count by class

Evidence: Monitoring script, two Evidently HTML reports in `monitoring/evidently_reports/`, and a Prometheus scrape screenshot.

3.7 Component 7: Documentation

1. **README.md** — project title, architecture overview, and a **Quickstart** that lets a reviewer run the serving app in **three commands or fewer**
2. **Model Card** (`docs/model_card.md`) — description, intended use, training data, metrics (overall and per subgroup), limitations, ethical considerations
3. **Data Card** (`docs/data_card.md`) — source, schema, preprocessing decisions, known biases, privacy and licensing notes
4. **Experiment Log** — MLflow runs exported as `docs/experiment_log.csv` (all runs, parameters, metrics)

Evidence: All four documents present at the paths above. The Model Card and Data Card will be read during the final discussion.

3.8 Component 8: Project Setup & Reproducibility

- A `requirements.txt` with **pinned versions** (`package==x.y.z`) for every dependency
- A central `configs/params.yaml` file that holds **all** pipeline parameters (no hardcoded values in any source file)
- A `README.md` Quickstart section with step-by-step instructions to install dependencies, run the pipeline, and start the serving app — **tested on a clean environment**
- A `.gitignore` that excludes data files, model artifacts, virtual environments, and IDE config (raw data must never be committed to Git)

Evidence: `requirements.txt`, `configs/params.yaml`, `.gitignore`, and the `README` Quickstart verified by a clean install.

4 Bonus Components

Two independent bonus components are available. Teams may attempt one or both.

Teams that containerise their serving application with Docker earn up to **10 bonus points**.

Requirements for full bonus (10 points):

- A **Dockerfile** for the serving application following best practices: pinned base image, non-root user, minimal image size
- A **docker-compose.yml** orchestrating at least **two services**: the serving app and the MLflow tracking server (or monitoring service)
- `docker compose up --build` starts the serving app with **zero manual steps**; the `/health` endpoint must respond
- All inter-service communication via Docker internal networking (service name as host-name, not `localhost`)

Partial credit (5 points): A working Dockerfile that builds and runs the serving app as a standalone container (no Compose required).

Teams that implement training pipeline orchestration with **Apache Airflow** or **Prefect** earn up to **10 bonus points**.

Requirements for full bonus (10 points):

- A DAG (Airflow) or Flow (Prefect) encoding the **full training pipeline**
- At least **five tasks**: `validate_data`, `preprocess`, `train`, `evaluate`, `register_model`
- DAG is **schedulable** — demonstrate a triggered run via the UI
- Task failure halts all downstream tasks with a clear error log

Partial credit (5 points): A working DAG/Flow with at least three tasks, functional but not fully integrated with the pipeline.

5 Deliverables & Submission

5.1 What to Submit

Submit all four items by **Friday, 8 May 2026 at 11:59 PM** via the Moodle portal.

#	Deliverable	Details
1	GitHub Repository	Public or private repo with instructor added as collaborator. Must contain all code, configs, <code>.dvc</code> files, Dockerfile, <code>docker-compose.yml</code> , <code>.github/workflows/</code> , and docs. Do not commit raw data files — use DVC.
2	Technical Report (PDF)	10–15 pages covering: Introduction & Dataset, Preprocessing & Feature Engineering, Experiments & Results, Pipeline Architecture, Monitoring Results (both Evidently reports), Challenges & Lessons Learned, Conclusion & Future Work.
3	Demo Video	5–8 minute recording showing: (a) starting the serving application (locally or via Docker if applicable), (b) a live prediction via <code>/predict</code> , (c) the MLflow UI with multiple experiment runs, (d) the Evidently drift report in a browser. Upload to YouTube (unlisted) or Google Drive and paste the link in the report and Moodle form.
4	Moodle Submission Form	GitHub URL, video URL, dataset name, and all three team members' full names and student IDs.

5.2 Required Repository Structure

```

project-root/
├── .github/
│   └── workflows/
│       └── ci.yml
├── data/
│   ├── raw/           # tracked by DVC, NOT committed to Git
│   ├── processed/     # tracked by DVC
│   └── splits/        # tracked by DVC (train/test sets)
├── src/
│   ├── data/          # loading, validation, preprocessing
│   ├── features/       # feature engineering
│   ├── training/      # model training & HPO
│   ├── evaluation/    # metrics & model validation
│   └── serving/        # FastAPI application
├── monitoring/
│   ├── run_monitoring.py
│   ├── evidently_reports/
│   └── prometheus/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── data/

```

```
docs/
  model_card.md
  data_card.md
configs/
  params.yaml
docker-compose.yml      # optional (Bonus A)
Dockerfile              # optional (Bonus A)
dvc.yaml
dvc.lock
requirements.txt
README.md
notebooks/              # EDA only; no pipeline logic here
```

6 Grading Rubric

6.1 Component Score Breakdown

The total project score is **100 points** (= 35% of the final course grade). Up to **20 bonus points** are available across two optional components (Section 4): Docker containerisation (+10) and pipeline orchestration (+10).

Component	Points	Grading Criteria
1. Data Versioning (DVC)	10	Pipeline runs without error; three artifacts tracked; <code>dvc repro</code> is deterministic; remote configured and documented
2. Preprocessing Pipeline	12	sklearn Pipeline complete and serialised; all steps config-driven; artifact DVC-tracked; at least three unit tests passing
3. Experiment Tracking & Registry	15	≥ 3 experiments fully logged; HPO applied; best model registered and promoted to Production via API; metrics and artifacts complete
4. Model Serving (Flask or FastAPI)	15	<code>/health</code> and <code>/predict</code> endpoints working; model loads at startup; input validation present; test script passes
5. CI/CD (GitHub Actions)	13	All four CI stages passing; coverage $\geq 70\%$; branch protection active; green CI screenshot provided
6. Monitoring & Drift Detection	15	Two Evidently reports present; drift threshold logic implemented; drift simulation documented; Prometheus metrics operational
7. Documentation	10	README with Quickstart; Model Card and Data Card complete; MLflow experiment log exported
8. Project Setup & Reproducibility	10	Pinned <code>requirements.txt</code> ; all params in <code>configs/params.yaml</code> ; <code>.gitignore</code> correct; clean-install Quickstart verified
Total	100	
BONUS A: Docker Containerisation	+10	Dockerfile builds; Compose starts serving app; <code>/health</code> responds
BONUS B: Orchestration (Airflow/Prefect)	+10	DAG with ≥ 5 tasks; schedulable; failure handling shown

6.2 Quality Scale

Score	Description
Full marks	Fully implemented; runs without errors; follows lecture best practices
70–90%	Mostly complete; minor gaps (one test missing, a path hardcoded, etc.)
40–69%	Core logic present but significant gaps or key best practices ignored
1–39%	Attempted but largely broken; effort evident but result insufficient
0	Component missing or non-functional with no meaningful attempt

6.3 Individual vs. Team Score

The **base project score** (out of 100) is shared equally among all team members. However, **individual adjustments of up to ± 15 points** may be applied based on each member's demonstrated understanding during the May 13 discussion.

Questions may cover **any component**, including components coded by a teammate. Teams that cross-review each other's code during development consistently perform better.

7 Team Collaboration & Academic Integrity

7.1 Team Formation & Registration

Teams consist of exactly **three students**. Register your team on Moodle by **22 April 2026**. Changes to team composition after that date require written instructor approval.

7.2 Collaboration Guidelines

- All members must make **meaningful commits** to the shared repository. A member with zero or trivial commits may receive a reduced individual score
- **No predefined roles** — divide work as you see fit, but every member must understand the entire pipeline
- Use **pull requests** for integrating feature branches into **main**; at least one teammate must review each PR before merging
- Use **GitHub Issues** to track tasks, bugs, and discussions; the Issues tab will be reviewed as evidence of project management
- Adopt a **feature branching strategy**: `main` $\leftarrow$ `feature/xxx` via pull request

Team Advice: Review Each Other's Code

The most common source of low individual scores is working only on one part of the pipeline. Make deliberate time to walk through each other's implementations, ask questions, and understand every design decision.

7.3 Academic Integrity

- All code must be **written by the team**. Copying from other teams or previous cohorts is academic dishonesty and will result in a **zero for the project**
- **AI coding assistants** are permitted as productivity aids, provided every team member fully understands the generated code; you are accountable for everything in your submission
- **Cite all external resources**: tutorials, templates, or reference implementations must be acknowledged in the README or report
- Suspected violations will be escalated to the academic integrity committee

8 Technical Guidance

8.1 Getting-Started Checklist

Work through this checklist in order:

- ☐ Create a shared GitHub repository and add all team members as collaborators
- ☐ Set up a Python virtual environment; create `requirements.txt` with pinned versions from the start
- ☐ Run `git init` and `dvc init`; commit the resulting `.dvc/` directory
- ☐ Download your dataset; add it to DVC with `dvc add data/raw/`
- ☐ Start an MLflow tracking server locally; verify the UI at `http://localhost:5000`
- ☐ Train a baseline model and log it to MLflow
- ☐ Write a minimal Flask or FastAPI app with `/health` and `/predict`; verify it returns predictions
- ☐ Add a minimal `.github/workflows/ci.yml` running `pytest`; confirm it passes on GitHub
- ☐ (*Bonus A*) Write a `Dockerfile`; confirm the container builds and serves predictions; add `docker-compose.yml` if pursuing full bonus
- ☐ Generate a baseline Evidently report on the training data

8.2 Common Pitfalls

- **Committing data to Git** — track large files with DVC; add them to `.gitignore`
- **Hardcoding paths and thresholds** — centralise everything in `configs/params.yaml`
- **Training inside the API** — the serving app must only *load* and *run* models; training belongs in the pipeline
- **Writing tests last** — tests are graded; write them as you code
- **Starting Docker late (if pursuing Bonus A)** — networking and volume issues take time to debug; do not leave Docker to the last day
- **Shallow monitoring** — the drift threshold logic and documented interpretation of results earn full marks, not just generating an Evidently report

8.3 Quick Reference: Key Commands

Task	Command
Initialise DVC	<code>dvc init && dvc remote add -d myremote /path/to/remote</code>
Run the DVC pipeline	<code>dvc repro</code>
Start MLflow server	<code>mlflow server --backend-store-uri sqlite:///mlflow.db --port 5000</code>
Start the full stack	<code>docker compose up --build</code>
Run tests with coverage	<code>pytest tests/ --cov=src --cov-report=term-missing</code>
Send a test prediction	<code>curl -X POST http://localhost:8000/predict \</code> <code>-H "Content-Type: application/json" \</code> <code>-d '{"feature1": 1.2, "feature2": 0.5}'</code>
Generate drift report	<code>python monitoring/run_monitoring.py</code>